SHF: Energy-Efficient and Cost-Effective Architecture for Highly Reliable Memory Systems with Current and Emerging Technologies

1. Research Overview

Memory reliability has become a severe concern for contemporary memory systems. Two large-scale field studies in 2009 [39] and 2012 [42] reported that DRAM memories had much higher raw error rates than expected. More importantly, DRAM errors were found to be highly correlated. The findings raised a serious question about the effectiveness of memory error protection schemes used in product systems, because those schemes are designed against random errors. A later study in 2015 [41] not only confirmed the previous findings but also showed a worrisome trend of increasingly weakening DRAM reliability because of the advance in DRAM fabrication technology. The technology advance has drastically improved DRAM density, bandwidth, and energy efficiency, but unfortunately it has weakened DRAM cell reliability because of shrunk cell size.

The weakening memory reliability also has a serious consequence: it incurs a new type of vulnerability in computer security. The standing out issue is Rowhammer attack [25, 32], which exploits the weakening cell reliability of high-density DRAM devices. In such an attack, an adversary runs a malicious program to flip data bits in memory; and with a carefully crafted program, the adversary can obtain privileged access to the system [40]. This is a severe problem and surprisingly challenging to address. Many designs have been proposed, and JEDEC has made an extension to the DDR5 standard [19], to address the problem. However, it persists and even worsens, because DRAM cell reliability continues to weaken as DRAM fabrication technology advances [6]. The latter is a bless for memory capacity, bandwidth, and energy efficiency, but not for reliability.

The PI and co-PI propose a new memory architecture, called RAIMD (Redundant Array of Independent Memory Device), and will fully investigate the new architecture to address those challenges. RAIMD is similar to RAID (Redundant Array of Inexpensive Disks) in organization: memory devices in RAIMD are independent with local access scheduling, contain local error correcting codes, and may form parity groups for error correction. It supports strong reliability with low cost and very high energy efficiency. It is also highly flexible: a RAIMD memory can be dynamically configured as RAIMD-0, RAIMD-5, RAIMD-6 or other variants as done in RAID, with varying ratio of redundancy, to allow trade-offs between reliability and cost. It can offer much higher reliability than ECC memory and Chipkill Correct memory for a given storage budget, particularly when used with memory devices of weak reliability.

RAIMD is designed for reliability but can also enhance security and support new memory technologies. RAIMD supports strong end-to-end error detection that covers memory cells, data bus, address bus, and I/O pins and circuits, which prevents Rowhammer attacks from succeeding. It uses decoupled memory access scheduling, which allows memory devices to work semi-independently with the memory controller, and thus can easily support devices using NVM (non-volatile memory) technologies [27, 52, 38, 37]. Those technologies have been studied intensively for two decades; and some has been tried by the industry in product systems but unsuccessfully [44]. Because of this property, in the RAIMD architecture those memory devices can have complex internal circuits to enhance their durability and reliability, which are critical to their adoption.

RAIMD is not a trivial migration of RAID to memory. There is a critical difference between memory systems and storage systems: memory latency is measured in nanoseconds, while disk latency is measured in milliseconds or at least a fraction of milliseconds. Any increase of memory latency incurred by RAIMD

could have significant impact on program performance. We have conducted a preliminary study to address the performance concern as well as the energy saving potential of RAIMD. The initial result has been published in a short paper [46], and a full paper has been submitted and is under review. The evaluation shows that the average performance overhead of RAIMD-5, which resembles RAID-5 in organization, is only 3.8% over a DDR5 baseline for SPEC CPU2017 and CPU2006 workloads. It reduces the average memory energy consumption by 25.9% from the DDR5 baseline, which has no ChipKill protection. When compared to a ChipKill scheme for DDR5, it reduces the average energy consumption by 69.4%. The design is well crafted to eliminate any increase of memory read latency and to minimize the increase of memory traffic. It contains a set of holistic changes to memory controller, device interface, memory command protocol, memory address mapping, and last-level cache.

RAIMD has several other advantages as a general-purpose, energy-efficient, and highly reliable memory architecture. It allows a low-cost implementation suitable for personal computers and mobile devices: a RAIMD memory can be constructed using a small number of memory devices, while a conventional ChipKill design requires many memory devices to be activated to serve one request. Its high energy efficiency is not only desired on those computers but also on servers. It is also highly configurable: a RAIMD memory can be configured as RAIMD-0 (no error protection), RAIMD-5, RAIMD-6 (double ChipKill protection), or other variants; and the configuration is done at booting time, not manufacturing time. Therefore, it fits computer systems with very low to very high reliability requirements, and can be re-configured to just meet the reliability target.

We will conduct a comprehensive research to investigate the full potential of RAIMD. In particular, we will study RAIMD with decoupled memory access scheduling (*decoupled scheduling* in short), which has not yet been covered by the preliminary study. We believe it is the key to fully addressing the growing concern on memory reliability and durability, as well as to explore the opportunities of adopting new memory technologies. With decoupled scheduling, the memory controller issues activate, read/write, and other miscellaneous commands to memory devices, but memory devices can schedule the actual timing of the corresponding memory operations. By comparison, with rigid scheduling used in current memory systems, memory devices must respond to the memory controller as slave devices. The change is small but important: A memory device can now be contain customized enhancement circuits to address any reliability weaknesses caused by a particular memory technology or fabrication process. The problems of DRAM disturbance errors and PCM (Phase-Change-Memory) write endurance can be fully addressed in this approach. High-density devices with weakened reliability can be further enhanced with strong ECC code embedded in the devices. When combined with the RAIMD organization, a designer can build a highly reliable memory system with high energy efficiency and low cost.

A key issue, and the major challenge, is the scheduling of memory bus usage because the memory controller no longer has the full control of memory devices. We have conducted another preliminary study on decoupled scheduling for a system of hybrid DRAM and NVM memory modules [26]. The result is promising: it shows that decoupled scheduling may approach, and even exceed, the performance of rigid, centralized scheduling. However, decoupling in RAIMD is more challenging than in a simple hybrid memory system because the timings of memory operations in RAIMD will have much higher variance. We propose to use perceptron-based neural model to assist decoupled scheduling. Neural model based prediction has been shown to be effective for improving the accuracy of branch prediction [20, 4] and memory prefetching [15]. In RAIMD, it will be used to predict memory operation completion time (and device ready time) with decoupled scheduling, which we believe is a sound and promising approach.

The research agenda is as follows:

- To study the integration of decoupled memory access scheduling in the RAIMD architecture, and the

use of device-specific enhancement circuits to improve reliability, security, and durability of memory devices.

- To study neural model based prediction to improve the performance of decoupled scheduling for memory devices of timing diversity.
- To investigate holistic management of memory reliability, including the optimal configuration of global protection and local protection for a given storage budget.

2 Proposed Research

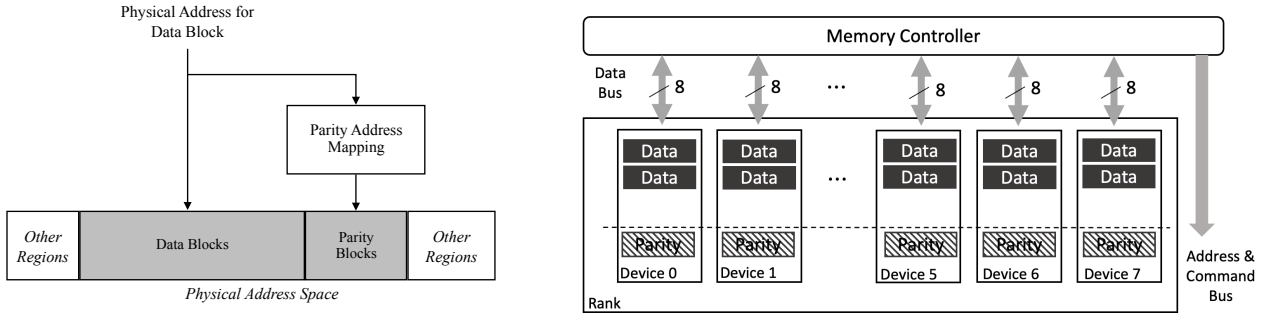
In this section, we first discuss the characteristics of RAIMD, present two preliminary studies on a baseline RAIMD design and decoupled memory access scheduling, respectively, and then discuss the research agenda items in detail.

2.1 Characteristics of RAIMD

RAIMD has a distinguishing characteristics, when compared to conventional memory architecture such as DDR4 and DDR5, that a memory request is served by a single memory device rather than a rank of devices. A memory block of LLC block size, typically 64 bytes or 512 bits, is mapped to a single device. It drastically reduces memory operational energy, i.e., energy spent on precharge and activation, at the cost of increased data transfer time. For example, in a DDR5-6400 system using x8 devices (with 8-bit data I/O), a memory block of 64 bytes is mapped to four memory devices, and it takes 2.5 ns to transfer the block on memory bus. In an equivalent RAIMD system, the block is mapped to a single device, and thus it takes 10 ns to transfer the block on memory bus. However, the latency of an idle memory is typically dozens of nanoseconds, and for memory-intensive workloads the dominant portion of memory latency is queueing delay. Thus, the increase of data transfer time may only have a moderate impact on performance. In RAIMD, a memory device is effectively a memory module by itself, and thus memory-level parallelism increases substantially, which may offset the performance loss due to increased memory latency.

Because of this characteristics, a RAIMD memory system is flexible in its organization because it can be configured dynamically during system booting time. Therefore, such a system allows trade-offs among performance, energy efficiency, cost, and reliability. To one extreme, it can be configured as RAIMD-0 (no redundancy) if reliability is not a concern for a given workload. To another extreme, RAIMD-1 (mirroring) offers very high reliability at the cost of reduced memory capacity. In practice, RAIMD-5 (with single parity block per group) and RAIMD-6 (with double parity blocks per group) are of great interest because they provide high reliability with relatively low cost. The parity-to-data ratio of them can be adjusted, giving another option of making trade-off. After all, all those RAIMD variants are highly energy-efficient when compared to a conventional memory architecture of wide rank.

A memory device in RAIMD is required to have extra bits per memory block to store an error detecting code (EDC). The requirement is consistent with the JEDEC DDR5 standard, which requires extra on-die ECC bits in DDR5 devices for local error detection and correction. However, EDC in RAIMD is generated and checked by the memory controller. Therefore, it is capable of end-to-end error detection that protects memory cells, I/O pins and circuits, data bus, and address bus. Note that for address bus error detection, memory address bits will be combined with data bits to generate the EDC. The memory block size in RAIMD is the same as the LLC block size, which is 512 bits (64 bytes) for most contemporary processors. Because of the relatively large block size, EDC in RAIMD is highly efficient. For example, a 32-bit EDC



(a) Address mapping and memory layout in the physical address space.

(b) Memory layout in the device address space.

Figure 1: The framework of address mapping and a generic memory layout.

can provide strong error detection for a 512-bit block, with a moderate storage overhead of 6.25%. A memory device may also contain additional ECC bits for local error correction (as DDR5 does), which further enhances the reliability. We will discus more in Section 2.5.

2.2 Preliminary Studies

We have conducted two preliminary studies related to the proposed RAIMD architecture. The first study is about a baseline design of RAIMD and its performance and energy efficiency. The result is promising: it shows that with careful design optimizations, RAIMD can support ChipKill protection with much higher energy efficiency than conventional memory architecture with a very moderate performance loss (less than 4%). The second study is related to decoupled scheduling of memory access, which is also promising: it shows decoupled scheduling for DRAM memories can achieve the same or even slightly better performance than conventional, rigid memory scheduling.

2.2.1 Baseline RAIMD Design

Overview. We have conducted a baseline design of RAIMD based on DDR5-6400 memory devices. The RAIMD design contains two critical features to maintain high memory speed and to reduce memory traffic. A critical design issue in RAIMD is the memory layout and address mapping: any complex address mapping from the physical address to the device address (with channel, rank, bank, row, and column components) may significantly increase memory latency. Note that it is not an issue to a storage system, as the latency of a storage device is orders of magnitude longer than that of a memory device. Furthermore, a system designer may prefer a particular address mapping scheme for a sound reason; for example, to use page-interleaving for spatial locality, cacheline-interleaving for parallelism, or a variant of their combination for a balance. This requirement further complicates the address mapping in RAIMD. Second, since RAIMD uses parity blocks for error correction, the increase of memory traffic for parity block access is a concern. RAID-5 has the small write problem [8]: a write may incur four disk accesses, namely data block read, parity block read, data block write, and parity block write. Fortunately, we have found a solution that not only eliminates parity block reads (except during error correction) but also significantly reduces parity writes, which we will discuss in detail.

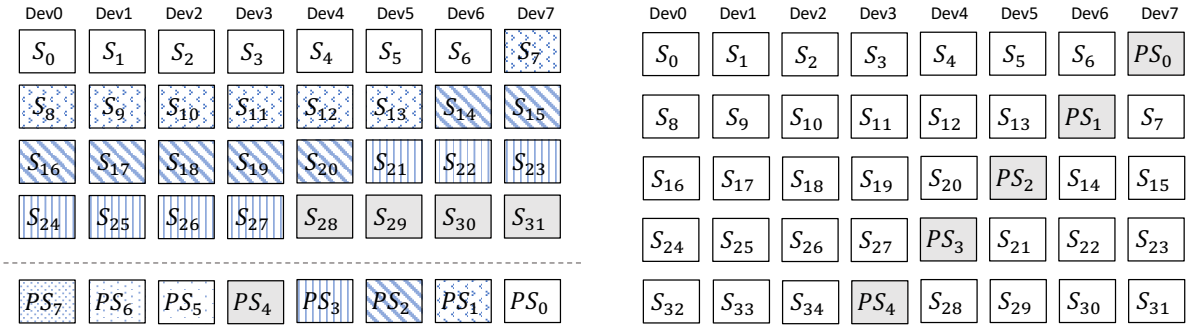


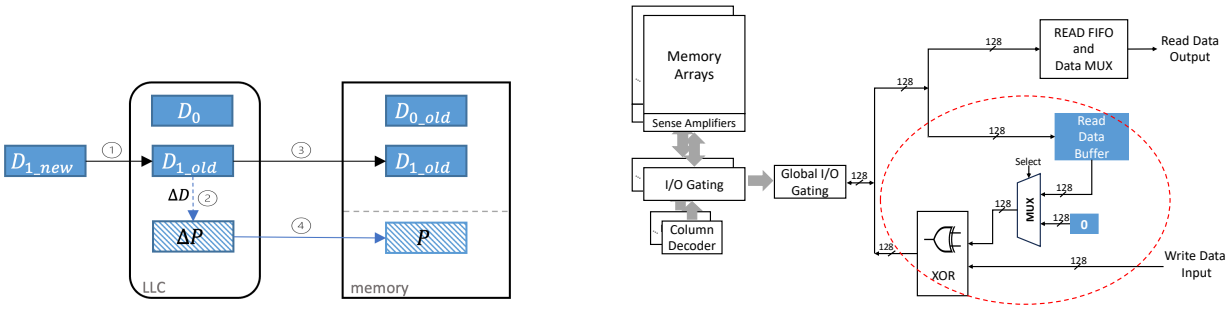
Figure 2: (a) The memory layout of a particular RAIMD-5 memory system for balanced row-buffer locality and bank-level parallelism. (b) The block layout of RAID-5 for comparison.

Address mapping and memory layout. RAIMD solves the problem by using dual address mappings for data blocks and parity blocks, as shown in Fig. 1. Data blocks and parity blocks are mapped to separate regions of the physical address space with different prefix address bits. The address mapping for data blocks only involves bit selection logic and optionally simple logic operations such as XORing. Thus, this is no extra latency for data block access compared to a conventional memory system. The address mapping for parity blocks, on the other hand, is complex to achieve a desired block layout. A Parity Address Mapping logic is applied to the physical memory address before it is mapped into the device address space (Fig. 1a). This logic may add a delay of multiple nanoseconds to parity block access, but it is not on the critical path of program execution and thus has negligible performance impact.

Fig. 2 shows the detail of address mapping and memory layout for a particular RAIMD-5 memory system of 1:7 parity-to-data ratio. It uses a stripe-based address mapping scheme to balance row-buffer locality and bank-level parallelism. A stripe is consecutive K blocks in the physical address space, where $K = 4$ by default. Fig. 2a shows the layout of data stripes and parity stripes, where S represents a data stripe and PS represents a parity stripe. For example, PS_0 is the parity stripe for S_0 to S_7, PS_1 is for S_7 to S_13, and so on. As the figure shows, the data and parity stripes in any parity group are evenly distributed to all devices, and thus there is no bottleneck for parity accesses. The RAIMD-5 layout is revised from the conventional RAID-5 layout, as shown in Fig. 2b, by moving the parity stripes down to the bottom. This movement simplifies the physical-to-device address mapping. It does not incur any performance or energy penalty in a memory system; that may not be true to a storage system of mechanic disks.

With the above address mapping and memory layout, RAIMD can continue to use conventional physical-to-device address mapping. It incurs no extra delay on data block access. Parity block access, on the other hand, has an extra delay because of the Parity Address Mapping (PAM) logic (Fig. 1a), which maps a data block address to the associated parity block address. The logic may involve shift, module, division, and multiplication logics and may take multiple clock cycles to complete. However, this extra delay is not on the critical path of the program execution, and thus it has negligible impact on performance. Thus, this framework allows a designer to use a complex address mapping to achieve a desirable memory layout, e.g., for a different parity-to-data ratio, stripe size, or having double parity blocks in a parity group. A memory controller may have multiple PAM logics to support different memory layouts.

Parity update mechanism. The second critical issue in RAIMD is the increase of memory traffic for parity block access. RAIMD uses a novel parity update mechanism to reduce this overhead, as shown in



(a) Overview of the parity update process.

(b) Modification to memory I/O circuit (circled).

Figure 3: Parity update mechanism in RAIMD.

Fig. 3. We assume that the LLC is a write-back cache, which is true for contemporary high-performance processors. The LLC and the memory controller are extended to support a new request called parity update, which is treated as a special type of write request. Upon a dirty write-back to the LLC, the XORed difference ΔD of the old data and the new data is generated (steps ① and ②), and the physical address of the associated parity block is generated (Fig. 1a). A parity update request is then generated and inserted into the write queue of the LLC. The data block will be written back to cache (step ③). Independently, ΔD will merged into a cache block for the parity block address if it exists; or written to a newly allocated cache block if otherwise (step ④). The parity block will cause a parity block update to memory when it is evicted from LLC.

To serve a parity update request to the memory, the memory controller may issue up to three memory commands, namely precharge, activate, and parity update. Memory devices need to support a new command called parity update. Its timing is the same as the write command. Fig. 3b shows the circuit modification inside a DDR5 x8 device to support the new command. We have done a thorough circuit analysis and concluded that the change does not affect the timing parameters related to the write command. That is because in modern memory access scheduling, there is a sufficient time interval between the arrival of a write command and the arrival of the write data such that the added circuit does not increase the critical path delay (except for the delay of the XOR gate, which is negligible).

With this parity update mechanism, parity read traffic is eliminated (except during error correction), and parity write traffic is significantly reduced because of caching. Our performance evaluation confirms that parity traffic is very moderate; the average increase of memory traffic is less than 5% over the baseline, and the average increase of cache misses is less than 2%. It is worth mentioning that a similar idea called partial parity caching has been proposed and studied to improve the performance and lifetime of flash-based RAID-5 systems [17, 9], but the implementation in RAIMD is very different.

Evaluation Results We have evaluated the performance and energy efficiency of multiple RAIMD variants using gem5 [5] and SPEC CPU2006 and CPU2017 workloads. We focused on a particular RAIMD-5 configuration of 1:7 parity-to-data ratio based on x8 DDR5-6400 devices. It uses a 32-bit error detection code per 512-bit data block to detect errors. The code is generated and checked by the memory controller and stored in memory device, and therefore the detection covers memory cells, bus, and I/O circuits. The result shows that RAIMD-5, which supports ChipKill protection, reduces memory energy consumption by 25.9% on average over a DDR5 baseline with no ChipKill protection. It incurs a moderate performance loss of 3.8% on average, which is mostly caused by increased data transfer time in memory access latency. When compared to unity ECC [22], a recent DDR5-based ChipKill design for servers, RAIMD-5 reduces

memory energy consumption by 69.4% on average. It is not a surprise, because unity ECC has to distribute the content of any ECC word to ten devices, and thus every memory access involves ten devices. In general, parity block based error correction is more efficient than ECC-based one for memory, because the former can use only one device to serve a memory request. The total storage overhead ratio of the above RAIMD-5 configuration is 20.54%, with 14.29% for parity and 6.25% for error checking code. By comparison, the storage overhead ratio of unity ECC is 31.25%, with 25% for extra ECC devices and 6.25% for on-die ECC.

A RAIMD system can be dynamically configured during system booting time as long as the PAM logic supports the needed address mapping. Our evaluation shows that a RAIMD-6 configuration, which supports double ChipKill protection, reduces memory energy consumption by 23.7% and incurs a performance loss of 4.3% when compared to the DDR5 baseline. A server computer may opt to use RAIMD-6 for the extra reliability. A consumer-level computer, on the other hand, may use RAIMD-5 with a smaller parity-to-data ratio to lower the storage overhead and thus reduce the cost.

2.2.2 Decoupled Memory Access Scheduling

The second preliminary study focused on decoupled memory access scheduling (decoupled scheduling in short thereafter) to support a heterogeneous memory system. In a conventional memory system, the memory controller conducts centralized scheduling of memory operations, including precharge, activate, read/write, auto-refresh, and other operations. All memory devices behave as slave devices to the memory controller. This was necessary because it avoids memory bus conflict and simplifies device circuit design. However, it is severe limitation to the adoption of emerging, non-volatile memory technologies, including PCM (Phase-Change Memory), STT-RAM (Spin-Torque-Transfer RAM), ReRAM (Resistive RAM), and MRAM (Magnetic RAM). They are promising alternatives to DRAM, but they behave differently and require careful management for longevity, reliability, performance, and/or energy efficiency. It is almost infeasible for a memory controller, which is an integrated part of a processor, to support those diverse technologies using centralized scheduling of memory operations.

We have proposed a form of decoupled scheduling called POMI (POlling-based Memory Interface) [26]. It is an improved design over UniMA, an earlier design of decoupled scheduling by the co-PI [14]. It is also closely related to Twin-Load [11], which supports a hybrid memory system of two types of memory modules with different latencies. POMI utilizes a device-specific buffer chip per memory module to implement a polling based memory bus protocol. The buffer chip receives read and write requests from the memory controller and performs local scheduling of device-specific operations, including precharge, activate, read/write. The memory controller conducts global scheduling of read and write requests and no longer tracks the status and ongoing operations at every memory module. The design not only reduces the complexity of the memory controller, but more importantly allows it to interoperate with memory modules using diverse technologies.

Decoupled scheduling, however, requires a new mechanism to avoid bus conflict. A new polling-based is proposed to communication between memory modules and the memory controller. When receiving a read request from the memory controller, the buffer chip of the target memory module sends a feedback with an estimated ready time. The memory controller schedules the bus usage, and it may send a transfer signal, which indicates bus availability, to a memory module based on its estimated completion time. The latter may perform data transfer after receiving the transfer signal, if the data is ready. Otherwise, if the approximation is wrong, it sends another feedback with a new estimated completion time. The evaluation shows that POMI can actually improve the average performance of memory-intensive workloads by 3.7% for a hybrid memory system of both DRAM and PCM. We found that polling adds an average time of 4.1 ns to memory latency, but it is more than offset by increased parallelism in memory modules because of local

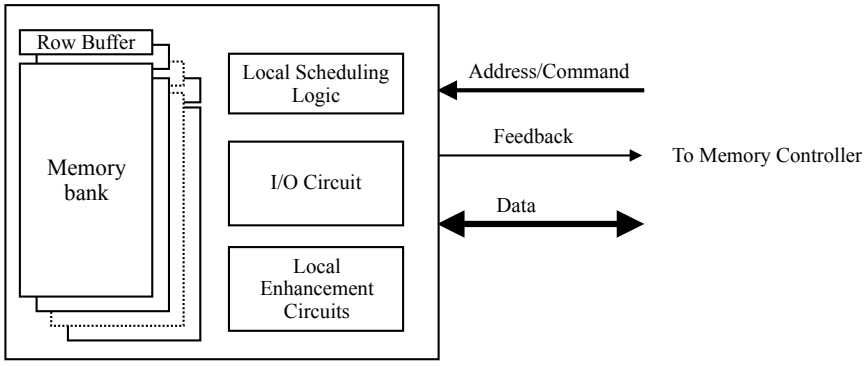


Figure 4: High-level view of the internal structure of a memory device with local scheduling.

scheduling.

The POMI design shows the potential of decoupled scheduling, but it may not be migrated to RAIMD directly. In RAIMD, local scheduling will be implemented into memory devices, not in a separate buffer chip. Memory device design is very sensitive to cost, and thus a more lightweight design is needed.

2.3 Decoupled Scheduling in RAIMD

The Mechanism. Decoupled scheduling is critical to the proposed RAIMD architecture. It allows the local enhancement for error protection and support for heterogeneous memory technologies. Fig. 4 shows a high-level view of a generic memory device in RAIMD with local scheduling and enhancement circuits. The memory controller issues three types of commands to a memory devices, namely activate, read/write, and inquiry. The former two commands play the same roles as in DDR4/DDR5 memories: an activate command moves a row of data to the row buffer, and a read/write command causes the read/write of a memory block from/to the row buffer. The read/write command contains a bit to indicate if the memory bank should be closed afterwards. The inquiry command is new, with which the memory controller may inquire if an activation has completed, if read data is ready in the bank, if the bank is ready for incoming write data, and other conditions.

Local scheduling is lightweight: its essential role is to decide the timing of memory operation for each command at each bank. It also responds to the inquiry command. The schedule logic maintains the status of each bank, enforce memory timing requirements (which decides the start time of an operation), and tracks the completion of the ongoing operation. In other words, the memory controller schedules the commands to devices, and a device schedules the timing of the corresponding operations. The memory controller schedules the bus usage to avoid bus conflict. It may only send a read command to a device if the data is ready in the row buffer, and a write command if the row buffer is ready to take on the incoming data. Finally, local scheduling logic also schedules other operations not directly related to memory requests, including refresh for DRAM devices.

The lightweight design ensures the cost effectiveness and operational efficiency of memory devices. There is no additional data buffer or queue, which would be expensive and hurt energy efficiency. Note that the design is different from POMI: the buffer chip of POMI is a separate chip and schedules memory commands to serve requests. In RAIMD, the latter is still the role of the memory controller. The local scheduling logic will be implemented as a small state machine. The cost is small in today's memory devices, whose I/O circuits are already a significant part of the chip area. The memory controller is also simplified,

as it no long tracks the status and ongoing operations of all memory devices. That can be costly if a memory controller has to support various memory technologies. It is worth mentioning that memory controllers are commonly embedded into processor chips, and thus this issue actually affects the interoperability of processor chips and memory devices.

Decoupled scheduling may have a moderate impact on performance because the inquiry and feedback process will add a delay to memory access latency. Our preliminary study on decoupled scheduling, however, indicate the impact is can be offset by increased memory parallelism, and decoupled scheduling may even improve performance. With decoupled scheduling, a memory command can be sent to the target memory device as soon as it is decided and the command/address bus is ready. With rigid scheduling, the memory controller must make sure the device will be ready for the operation when it receives the command (there are a number of timing constraints), otherwise it has to delay the command. The proposed research will give a quantitative answer to this question.

Local enhancement circuits. With decoupled scheduling, memory devices can have device-specific and technology-specific enhancement circuits for various purposes. For DRAM devices, those may include efficient DRAM refresh, Rowhammer defense, and local error correction. For NVM devices, those may include wearout management, cell fault tolerance, and enhanced error correction. Those issues are not directly related to reliability, but they are important and can be addressed in the RAIMD architecture.

DRAM refresh is increasingly a concern to the performance and energy efficiency of high-capacity DRAM devices because more and more DRAM rows have to be refreshed in DRAM refresh cycle, which is 64 ms under normal temperature by the JEDEC standard. A study has shown that, for memory-intensive workloads, DRAM refresh energy consumption can reach 50% of the entire memory energy consumption, and the system performance may degrade by up to 30% [3]. An promising approach is retention-time aware DRAM refreshing [16, 29, 28, 35], which refreshes strong and weak rows at different rate. Most DRAM rows are strong rows, with retention time much longer than 64 ms. The problem in a conventional memory system, however, is that the memory controller has to manage the extra complexity and schedule fine-grain refresh commands to all memory devices. In RAIMD, it can be efficiently implemented as a local enhancement circuit, which can maintain a record of a small number of week rows (by profiling at booting time) and issue the fine-grain refresh commands locally. Because of decoupled scheduling, those locally issued refresh commands will not cause any hard conflict with the incoming commands from the memory controller.

DRAM Rowhammer attack [25, 23] is increasingly a security concern as DRAM manufacturing moves to more advanced fabrication technology. In this attack, an adversary may flip bits in a DRAM row by frequently accessing its neighbor rows to cause disturbance errors. Many solutions have been proposed ([2, 48, 45] for a few examples), but they incur non-trivial performance and energy overhead and are still not strong enough. A new extension of JEDEC DDR5 standard has introduced an optional Rowhammer mitigation framework called Per Row Activation Counting (PRAC) [19], which embeds per-row activation count in DRAM device and uses a special RFM (refresh management) command. However, a recent study has found the performance loss of PRAC can reach over 80% for DRAM devices of very low disturbance threshold [6]. The cause is mostly the inefficiency of the memory controller issued RFM command: it may block a memory bank for hundreds of nanoseconds. In RAIMD, the refresh management can be done by a local enhancement circuit with much higher efficiency. A local enhancement circuit can issue the RFM command, and each instance of refresh management can be fine-grain and take much shorter time. Again, because of local scheduling, locally issued RFM commands will not cause any hard conflict with memory controller issued commands.

Wearout management is an important issue to NVM devices. Many such designs have been proposed. For

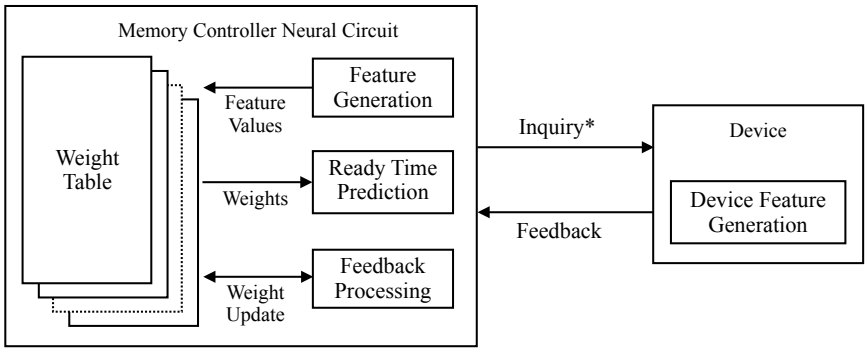


Figure 5: A high-level scheme of NM-RTP (neural model based ready-time prediction) for RAIMD.

example, an NVMD may contain an SRAM cache to reduce NVM writes, bit rotating design for wearout leveling, fault tolerant circuits to handle hardware errors, among many other design ideas [27, 52, 38, 37]. In a conventional memory system with rigid scheduling, it is infeasible to have a memory controller to support various NVMDs of those techniques, which requires the memory controller understand the internal of each device type. In RAIMD, the wearout management can be implemented as a local enhancement circuit, which schedules those operations when needed. Those operations, when ongoing, may delay activate and read/write operations, but again they will not cause a hard conflict in the scheduling.

We will study the integration of those designs into RAIMD as local enhancement circuits. Our focus is not on the circuit designs themselves, but on the efficiency of the integration and the performance impact. Some of those designs may cause a high variation to operation completion time, which is a concern to the overall performance of the baseline RAIMD. We continue to propose neural model based prediction to address this issue.

2.4 Neural Model Based Prediction for Decoupled Scheduling

The inquiry and feedback process of decoupled scheduling adds a delay to memory access latency, which we predict is a concern for memory devices with advanced enhancement circuits. We call them advanced devices thereafter. It is unlikely a concern for simple devices as our preliminary study [**?**] shows. However, advanced devices may have much higher timing variation in their operations than simple devices. For example, a DRAM device with Rowhammer defense will trigger internal target refreshes, which may block normal operations (activate, read/write, and associated precharge). An NVM device with wearout leveling circuits may trigger internal data movement, which may block normal operations temporarily. Even for simple devices, internal DRAM refresh may cause delay to normal operations, which was not considered in the preliminary study as it uses centralized scheduling for DRAM refreshes. As the result, the overhead from the inquiry-and-feedback process will increase. The impact will be quantified in the proposed research, but new methods should also be investigated.

To improve the accuracy of ready-time prediction (RTP), we propose to use neural model based read-time prediction (NM-RTP) in RAIMD. Neural model with online learning has been used successfully in branch prediction [**?**] and memory prefetching [**?**]. We are also studying the use of a perceptron-based neural model to improve the accuracy of memory prefetching using memory-side information [**?**]. We believe a good neural model, particularly perceptron-based, is promising to improve RTP accuracy, but it has to be done carefully. Fig. 5 illustrate the NN-RPT scheme that we propose. The "Inquiry*" signal includes activate, read/write, and inquiry commands, for all of which the device will send back a feedback. The feedback

includes an estimated ready time and a feature value, the latter of which will be used as a device input to the neural circuit. A memory device contains a circuit for device-specific feature generation, which is expected to have a good correlation with the actual ready time. It should indicate how reliable the estimated ready time is, and provide a hint about the actual ready time if the estimate is wrong. The neural circuit will learn from all the feedbacks from the device, and then make a best prediction of the ready time, which assits the memory controller to schedule memory commands.

The majority of the cost lies in the weight tables of the neural circuit. It is important that the neural circuit is located in the memory controller, not in the memory devices. On the other hand, the circuit of device feature generation must reside in the device, and it should be customized for each device type so that the feature value is highly correlated to the ready time. The feature generation is important. For DRAM devices, it can be a hint for incoming or ongoing DRAM refresh, Rowhammer defense operations, low-power mode and mode switch, and others. For NVM devices, it can be a hint for incoming or ongoing wearout leveling operations, fault tolerance related operations, chance for buffer hits (if a performance-enhancing buffer exits), and so on. The whole scheme is a form of distributed learning circuit design: the neural circuit in the memory controller carries out most of the learning, and the circuit in the device provides its best assistantship. Finally, the neural circuit may also include other inputs, either from the inside of the memory controller or from the processor cores, as secondary features. In the proposed research, we will investigate the full detail the NM-RTP scheme, evaluate its performance, and optimize it to improve the accuracy.

2.5 Holistic Protection and Reliability Measurement Methodology

RAIMD provides a form of global protection: when an memory error is detected by the error checking code, the content of the faulty block is reconstructed by the blocks in other memory devices in the same parity block. It can correct any single-block error. However, given the worsening raw memory reliable, the probability of double-block error cannot be ignored, particularly for servers that require very high reliability. RAIMD-6 can correct double-block error, but it increases the cost and is less energy-efficient than RAIMD-5. After all, triple-block error may also become a concern for servers, which cannot be corrected by RAIMD-6.

RAIMD can be enhanced with local protection by using embedded ECC. The PI has studied embedded ECC for DRAM memory, in which ECC bits are stored with the associated data bits in the same DRAM row [7]. It is similar to on-die ECC of DDR5 but is more advanced. Embedded ECC has a unique advantage over SECDED ECC memory: it can use very long ECC word size. This is important, because the storage efficiency of ECC will increase drastically. Assume the data part of the ECC word is 512 bits, and the BCH code [**?**] is used as the codec. It takes only 10 ECC bits to correct a single-error, 20 ECC bits to correct a double-error, and 30 ECC bits to correct a triple-error. The storage overhead ratio is 1.95%, 3.90%, and 5.86%, respectively. It is much lower than the 12.5% overhead of SECDED ECC using an ECC word of 64 data bits and 8 ECC bits, which can only correct single-error. The BCH circuit can be part of the local enhancement circuits. It is worth mentioning that the ECC word size is limited by the row buffer size, not the LLC block size, and thus the data part of the ECC word can be increased to 1024 bits or longer for even better storage efficiency, though the cost of the ECC circuit will also increase. There is a large design space on local error correction, including the choice of the codec, the ECC word size, the design of the ECC circuit, and the performance and energy efficiency evaluation. We will fully investigate the design space.

Furthermore, the global protection and local protection together form a type of holistic error protection, and they can be balanced to maximize the protection strength for a given storage budget. An uncorrectable error will happen to RAIMD-5 if and only if two or more blocks in the same parity group have memory faults

that cannot be corrected by embedded ECC. Therefore, the holistic protection is much stronger than a baseline RAIMD-5, and it can be more efficient than RAIMD-6 in storage overhead and energy consumption. If extremely high protection is required, e.g. for devices of weak raw reliability, RAIMD-6 with embedded ECC can also be used. With the objective to maximize reliability for a given storage budget, there is another large design space to explore, including the choice of RAIMD-5 and RAIMD-6, the choice of parity-to-data ratio, and the choice and configuration of embedded ECC. We will also fully investigate those issues.

Finally, we will study new reliability measurement methodology to evaluate and quantify the reliability of RAIMD systems. It is necessary for searching the optimal configuration of the holistic protection, but it is challenging given the complexity of RAIMD systems and the complexity of memory fault patterns. It is impractical to use either pure analytical method or pure Monte Carlo simulation. A recent study introduces a new methodology to conduct empirical analysis of DRAM fault model; and has developed a fault and error simulator to evaluate the effectiveness of existing ECC schemes [21]. The simulation framework runs Monte Carlo trials to evaluate the reliability of ECC schemes, using error patterns produced from the fault model. It is unique in that it actually executes the ECC function in the Monte Carlo trials, which helps improve the evaluation accuracy for complex ECC schemes. We plan to extend this methodology and the simulation framework to evaluate and quantify the reliability of RAIMD systems. The challenge is that RAIMD systems are much more complex than ECC schemes, particularly when local error protection is used. Thus, the error patterns must be generated with a sound methodology; otherwise, either the Monte Carlo simulation takes excessive time to finish, or the reliability measurement will not be accurate. We will study this methodology in depth.

3 Related Work

ECC and ChipKill Memories As memory systems become more prone to multi-bit errors, various techniques have been proposed and applied to protect against memory errors. Redundant chips can be added to the DIMM to provide chip-level or rank-level protection [12, 30, 18, 13, 10, 22]. ChipKill has been proposed to correct single-chip errors and detect double-chip errors [13] by distributing ECC bits across multiple memory chips ensuring that a failure in a single chip affects only one bit of any ECC word. However, current schemes for chip-level protection either come with high performance overhead or low energy efficiency. A recent study proposes Unity ECC [22], which utilizes Rank-Level ECC (RL-ECC) for DDR5 memory. Two redundant x4 devices holding ECC bits along with eight x4 devices holding data bits are connected to a sub-channel. The eight ECC bits along with the 32 data bits are transferred on the 40-bit wide data bus to provide rank-level protection. This will result in high storage overhead, 32.8% with x4 devices assuming on-die ECC bits take up 6.25% as redundant cells [22]. It also incurs at least 25% energy overhead since each DRAM access will access eight data chips and two redundant chips. DDR5 has not yet received a widely accepted ChipKill-level ECC proposition [24] due to the split channels. We use this design for comparison in our evaluation of RAIMD because it is a recently proposed, DDR5-based design.

There are also non-conventional ChipKill correct solutions. For example, LOT-ECC [43] provides ChipKill correct capability using nine-device rank (assuming x8 devices), but with an increased ratio of ECC bits. Virtualized ECC [47] supports flexible error-correcting codes and stores data and ECC bits in separate locations. As another ChipKill alternative, Nair et al. propose XED [34] that utilizes on-die ECC to detect error and use DIMM-level parity to correct the error in a way similar to RAID-3.

RAID-Like memory systems There have been several existing studies using redundant memory with RAID-like layouts, such as Intel Memory Mirroring [36] with RAID-1 layout, IBM RAIM [31] with RAID-

3 layout, and RAIM5 [51] with RAID-5 layout. Intel Memory Mirroring stores a backup copy for the entire memory channel [36]. IBM RAIM provides comprehensive error protection against channel failures as well as bus failures and full DIMM and chip failures [31]. However, such strong protection comes with very high cost. It uses four memory channels with ECC and one channel with parity data, which results in nearly 40% storage overhead. Each memory read or write accesses 256 bytes across five memory channels simultaneously, which has a negative performance impact and significant energy overhead. Furthermore, it requires an advanced memory buffer (AMB) to handle the upstream and downstream data. Another study named RAIM5 [51] proposed an optimized RAIM configuration, which incurs 17% off-chip traffic overhead with 28% DRAM energy overhead. Another work named XED [33] uses RAID-3 layout. It can recover single-chip failure using the parity data stored in a ninth chip. The chip failure is detected by the on-die ECC; and a pre-defined "catch-word" is sent to the memory controller. As with the other designs, XED consumes more energy than conventional memory.

Sub-rank organizations The baseline RAIMD design is closely related to sub-rank memory organizations, such as Mini-Rank [50] and Multicore DIMM [1], to achieve high energy efficiency. For instance, Mini-Rank [50] divides each 64-bit rank into multiple sub-ranks and activates only a sub-rank of devices for each request. The Mini-Rank design does come with a performance penalty because the data transfer time for a memory block, typically 64 bytes, is reciprocal of the sub-rank size. Thus, the original Mini-Rank study (based on DDR3-1066) found that dividing a rank into two mini-ranks, e.g., four x8 devices per mini-rank, achieves the best energy-performance trade-off for most applications. However, with the current trend of ever-increasing DRAM data rate, it is now feasible to use a sub-rank of a single device, e.g., one x8 device per mini-rank. This change not only further improves energy efficiency but also enables error protection by the RAIMD design. Even though in Mini-Rank, ECC bits can be used and are distributed across different mini-ranks as in RAID-5, the ECC bits are not embedded with the data bits as on-die ECC, thus each memory access will need to access extra chips in one mini-rank, which is not efficient and wastes the memory bandwidth. In RAIMD, we address this issue by using parity data to form a RAID-5 like architecture, and on-die ECC to detect errors, forming strong error protection and detection against chip failures, but also achieving energy efficiency at the same time.

Earlier studies on decoupled memory access scheduling Regarding the memory access protocol design, a recent work called Twin-load [11] allows one processor to support both direct-attached and buffer-on-board memory with minimal changes to the existing DDR*x* protocol by splitting one read request to two loads. It changes the access protocol slightly, but not as comprehensive as the proposed design. In an earlier study, the PIs have proposed a framework, called UniMA [14], to support heterogeneous memory modules in one system by extending the function of bridge chip and offloading the scheduling of device-level operations to each memory module. It uses a token-based protocol for the bus communication between memory controller and modules. However, the bridge chip does not have any advanced functionaries, and the access protocol is different and the interface is more complex.

4 Results from Prior NSF Support of Last Five Years

The PI and co-PI have not been supported by NSF for the last five years.

5 Project Management Plan

The PIs plan to carry out the research with the following three-year plan:

- **Year 1.**

  We will focus on decoupled scheduling and NM-RTP. On decoupled scheduling, we plan to do Verilog modeling for the scheduling logic in memory device to verify its functional correctness and to evaluate its circuit cost, extend our current gem5-based simulation for RAIMD to model decoupled modeling, and run simulation to evaluate its performance impact. On NM-RTP, we plan to develop the neural model for RTP, do Verilog modeling for the neural circuit in memory controller and the feature generation circuit in device, verify their functional correctness, and evaluate their circuit cost and performance. We will prepare a paper on decoupled scheduling in RAIMD for conference publication.

- **Year 2.**

  We will focus on NM-RTP evaluation, local error correction with embedded ECC, and reliability measurement methodology. On NM-RTP, we plan to extend our simulation to model the neural model and its integration into decoupled scheduling, and optimize the neural model based on the evaluation result. On the latter two parts, we plan to develop new versions of embedded ECC, develop methods to measure their reliability, and study the integration of global error protection and embedded ECC. We will also develop Verilog modeling for embedded ECC circuits to evaluate its circuit cost and performance. On publication, we plan to publish two or three conference papers based on the results.

- **Year 3.**

  We will focus on holistic error protection and the continuous development of reliability measurement methodology. On holistic error protection, we will focus on its integration with embedded ECC and the balance of the two to maximize the reliability for a given budget of storage overhead. We will also develop new reliability measurement methods to guide and evaluate the integration. On publication, we plan to publish another two or three conference papers and prepare for journal publications. We also plan to finalize and publish our Verilog modeling and gem5-based simulation code on GitHub.

6 Education Plan

The educational impact of this project will be significant from the following aspects at the public university:

- **New curriculum development.** The proposed research will contribute to the current development of new graduate topic classes related to memory systems at the Department of Electrical and Computer Engineering at University of Illinois Chicago (UIC). The findings and simulation tools developed from this research will be introduced in the classrooms in a timely manner.

- **Undergraduate participation.** We will encourage highly motivated undergraduate students to participate in the proposed research. The College of Engineering at UIC has a summer intern program that encourages undergraduate students to work at research labs. We have been actively participating in this program. The students will be selected from undergraduate classes in computer organization and architecture, where we are major instructors for those classes.

- **Woman and under-representative minority student participation.** University of Illinois Chicago is one of the nation's most diverse public research universities and is a federally designated Minority Serving Institution. Being a woman faculty, the co-PI is especially committed to increasing the number of women pursuing advanced degrees in computer science and engineering. The PI has been

working with a woman PhD student in preliminary study of RAIMD, have advised multiple undergraduate underrepresented minority students. We will continue to expand outreach efforts to broaden the participation of underrepresented minority students in research and higher education.

7 Broader Impact, Dissemination Activities and Technology Transfer

The proposed research will lead to a new memory architecture to build memory systems that are highly reliable, energy-efficient, and cost-effective. It works for all general-purpose computers, including servers, personal computers, and mobile devices. It is also promising for emerging memory technologies, as those technologies tend to be less reliable than current DRAM technology. In the long run, the research will have a major impact on the design of computer systems. The proposed educational component will enrich computer system education related to memory system design for both undergraduate and graduate students.

The results of this project will be widely disseminated to the research community as well as practitioners through (i) technical publications in high quality journals/conferences and presentations at relevant conferences; (ii) open sourcing of the source code of our simulation tools; and (iii) tutorials on these techniques at conferences and companies that have related business.

This proposed research is of great practical interest to the industry. Our past research and designs on memory systems have been adopted in processors and chipsets [49]; and we expect effective technology transfer for the proposed research.

References

[1] J. H. Ahn, N. P. Jouppi, et al. Future scaling of processor-memory interfaces. In *Proceedings of SC'09*, 2009.

[2] Z. B. Aweke, S. F. Yitbarek, R. Qiao, R. Das, M. Hicks, Y. Oren, and T. Austin. Anvil: Software-based protection against next-generation rowhammer attacks. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 743–755, 2016.

[3] I. Bhati, Z. Chishti, S.-L. Lu, and B. Jacob. Flexible auto-refresh: Enabling scalable and energy-efficient DRAM refresh reductions. In *Proceedings of the 42Nd Annual International Symposium on Computer Architecture*, 2015.

[4] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez. Perceptron-based prefetch filtering. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 1–13, 2019.

[5] N. Binkert, B. Beckmann, et al. The gem5 simulator. *ACM SIGARCH computer architecture news*, 39(2):1–7, 2011.

[6] O. Canpolat, A. G. Yaglikci, G. F. Oliveira, A. Olgun, N. Bostanci, I. E. Yuksel, H. Luo, O. Ergin, and O. Mutlu. Chronus: Understanding and Securing the Cutting-Edge Industry Solutions to DRAM Read Disturbance . In *International Symposium on High Performance Computer Architecture*, pages 887–905, 2025.

[7] L. Chen, Y. Cao, and Z. Zhang. E3CC: A memory error protection scheme with novel address mapping for sub-ranked and low-power memories. *ACM Transactions on Architecture and Code Optimization*, 10(4):32:1–32:22, Dec. 2013.

[8] P. M. Chen, E. K. Lee, et al. RAID: High-performance, reliable secondary storage. *ACM Comput. Surv.*, 26(2):145–185, jun 1994.

[9] C.-C. Chung and H.-H. Hsu. Partial parity cache and data cache management method to improve the performance of an SSD-based RAID. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 22(7):1470–1480, 2014.

[10] K. Criss, K. Bains, et al. Improving memory reliability by bounding DRAM faults: DDR5 improved reliability features. In *Proceedings of the International Symposium on Memory Systems*, 2021.

[11] Z. Cui, T. Lu, S. A. McKee, M. Chen, H. Pan, and Y. Ruan. Twin-load: Bridging the gap between conventional direct-attached and buffer-on-board memory systems. In *Proceedings of the Second International Symposium on Memory Systems*, pages 164–176, 2016.

[12] T. J. Dell. *A white paper on the benefits of chipkill-correct ECC for PC server main memory*, 1997.

[13] T. J. Dell. A white paper on the benefits of chipkill-correct ECC for PC server main memory. *IBM Microelectronics Division*, 11(1-23):5–7, 1997.

[14] K. Fang, L. Chen, Z. Zhang, and Z. Zhu. Memory architecture for integrating emerging memory technologies. In *Proceedings of the 12th International Conference on Parallel Architectures and Compilation Techniques*, 2011.

[15] E. Garza, S. Mirbagher-Ajorpaz, T. A. Khan, and D. A. Jiménez. Bit-level perceptron prediction for indirect branches. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 27–38, 2019.

[16] T. Hamamoto, S. Sugiura, and S. Sawada. On the retention time distribution of dynamic random access memory (DRAM). *IEEE Transactions on Electron devices*, 45(6):1300–1309, 1998.

[17] S. Im and D. Shin. Flash-aware RAID techniques for dependable and high-performance flash memory ssd. *IEEE Transactions on Computers*, 60(1):80–92, 2011.

[18] Intel. Intel E7500 chipset MCH Intel x4 single device data correction (x4 SDDC) implementation and validation, 2002.

[19] JEDEC. JESD79-5c: DDR5 SDRAM Standard, 2024.

[20] D. A. Jiménez and C. Lin. Dynamic branch prediction with perceptrons. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture*, page 197, 2001.

[21] J. Jung and M. Erez. Predicting future-system reliability with a component-level dram fault model. In *Proceedings of International Symposium on Microarchitecture*, 2023.

[22] D. Kim, J. Lee, et al. Unity ECC: Unified memory protection against bit and chip errors. In *Proceedings of SC'23*, 2023.

[23] J. S. Kim, M. Patel, A. G. Yaglikci, H. Hassan, R. Azizi, L. Orosa, and O. Mutlu. Revisiting rowhammer: An experimental analysis of modern DRAM devices and mitigation techniques. In *Proc. of the 47th Intl. Symposium on Computer Architecture*, pages 638–651, 2020.

[24] M. J. Kim, M. Wi, et al. How to kill the second bird with one ECC: The pursuit of row hammer resilient DRAM. In *Proceedings of International Symposium on eicroarchitecture*, 2023.

[25] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. In *Proceeding of the 41st Annual International Symposium on Computer Architecuture*, pages 361–372, 2014.

[26] T. Le, Z. Zhang, and Z. Zhu. Polling-based memory interface. *ACM Trans. Des. Autom. Electron. Syst.*, 28(3), May 2023.

[27] B. C. Lee, E. Ipek, O. Mutlu, and D. Burger. Architecting phase change memory as a scalable dram alternative. In *Proceedings of the 36th annual international symposium on Computer architecture*, pages 2–13, 2009.

[28] J. Liu, B. Jaiyen, Y. Kim, C. Wilkerson, and O. Mutlu. An experimental study of data retention behavior in modern DRAM devices. volume 41, page 60, 2013.

[29] J. Liu, B. Jaiyen, R. Veras, and O. Mutlu. RAIDR: Retention-aware intelligent DRAM refresh. In *Proceedings of the 39th Annual International Symposium on Computer Architecture*, pages 1–12, 2012.

[30] D. Locklear. Chipkill correct memory architecture, 2000.

[31] P. J. Meaney, L. A. Lastras-Montano, et al. IBM zEnterprise redundant array of independent memory subsystem. *IBM Journal of Research and Development*, 56(1.2):4:1–4:11, 2012.

[32] O. Mutlu. The rowhammer problem and other issues we may face as memory becomes denser. In *Proceedings of Design, Automation & Test in Europe Conference & Exhibition*, pages 1116–1121. 2017.

[33] P. J. Nair, V. Sridharan, et al. XED: Exposing on-die error detection information for strong memory reliability. In *Proceedings of International Symposium on Computer Architecture*, 2016.

[34] P. J. Nair, V. Sridharan, and M. K. Qureshi. Xed: Exposing on-die error detection information for strong memory reliability. In *Proceedings of the 43rd International Symposium on Computer Architecture*, pages 341–353, 2016.

[35] M. Patel, J. S. Kim, and O. Mutlu. The reach profiler (reaper): Enabling the mitigation of dram retention failures via profiling at aggressive conditions. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pages 255–268, 2017.

[36] A. Patil, V. Nagarajan, et al. Dvé: Improving DRAM reliability and performance on-demand via coherent replication. In *Proceedings of International Symposium on Computer Architecture*, 2021.

[37] M. K. Qureshi, M. Franceschini, and L. Lastras. Improving read performance of phase change memories via write cancellation and write pausing. In *Proceedings of the 16th International Symposium on High Performance Computer Architecture (HPCA)*, pages 1–11, 2010.

[38] M. K. Qureshi, V. Srinivasan, and J. A. Rivers. Scalable high performance main memory system using phase-change memory technology. In *Proceedings of the 36th annual international symposium on Computer architecture*, pages 24–33, 2009.

[39] B. Schroeder, E. Pinheiro, et al. DRAM errors in the wild: a large-scale field study. In *Proceedings of the 11th International Joint Conference on Measurement and Modeling of Computer Systems*, 2009.

[40] M. Seaborn and T. Dullien. Exploiting the DRAM rowhammer bug to gain kernel privileges How to cause and exploit single bit errors Mark Seaborn and Thomas Dullien Bit flips !

[41] V. Sridharan, N. DeBardeleben, S. Blanchard, K. B. Ferreira, J. Stearley, J. Shalf, and S. Gurumurthi. Memory errors in modern systems: The good, the bad, and the ugly. page 297–310, 2015.

[42] V. Sridharan and D. Liberty. A study of DRAM failures in the field. In *Proceedings of SC'12*, 2012.

[43] A. N. Udipi, N. Muralimanohar, R. Balsubramonian, A. Davis, and N. P. Jouppi. LOT-ECC: localized and tiered reliability mechanisms for commodity memory systems. In *Proceedings of the 39th International Symposium on Computer Architecture*, pages 285–296, 2012.

[44] D. Waddington, M. Kunitomi, C. Dickey, S. Rao, A. Abboud, and J. Tran. Evaluation of intel 3d-xpoint nvdimm technology for memory-intensive genomic workloads. In *Proceedings of the International Symposium on Memory Systems*, MEMSYS '19, pages 277–287, 2019.

[45] Y. Wang, Y. Liu, P. Wu, and Z. Zhang. Discreet-para: Rowhammer defense with low cost and high efficiency. In *Proceedings of International Conference on Computer Design*, pages 433–441, 2021.

[46] P. Wu, T. Le, Z. Zhu, and Z. Zhang. Redundant array of independent memory devices. *IEEE Computer Architecture Letters*, 22(2):181–184, July 2023.

[47] D. H. Yoon and M. Erez. Virtualized and flexible ECC for main memory. In *Proceedings of the 15th International Conference on Architecture Support for Programming Languages and Operating Systems*, pages 397–408, 2010.

[48] J. M. You and J.-S. Yang. MRLoc: Mitigating row-hammering based on memory locality. In *Proc. of the 56th Design Automation Conference*, pages 1–6, 2019.

[49] Z. Zhang, Z. Zhu, and X. Zhang. A permutation-based page interleaving scheme to reduce row-buffer conflicts and exploit data locality. In *Proceedings of the 33rd Annual ACM/IEEE International Symposium on Microarchitecture*, pages 32–41, 2000.

[50] H. Zheng, J. Lin, et al. Mini-rank: Adaptive DRAM architecture for improving memory power efficiency. In *Proceedings of International Symposium on Microarchitecture*, 2008.

[51] R. Zheng and M. C. Huang. Redundant memory array architecture for efficient selective protection. In *Proceedings of International Symposium on Computer Architecture*, 2017.

[52] P. Zhou, B. Zhao, J. Yang, and Y. Zhang. A durable and energy efficient main memory using phase change memory technology. In *Proceedings of the 36th Annual International Symposium on Computer Architecture*, pages 14–23, 2009.